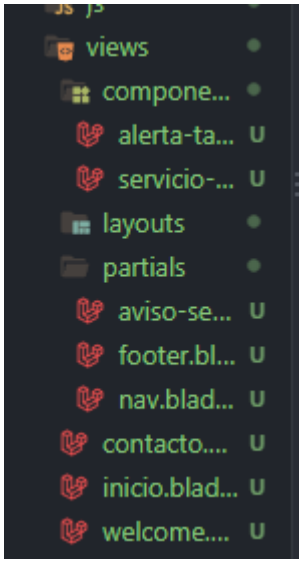
Taxi Fernán Núñez

La página principal presenta los servicios disponibles y un formulario permite enviar una solicitud de reserva. Restricciones de la práctica: no hay controladores ni base de datos. Todas las peticiones se resuelven con funciones anónimas en routes/web.php.

Instrucciones de arranque

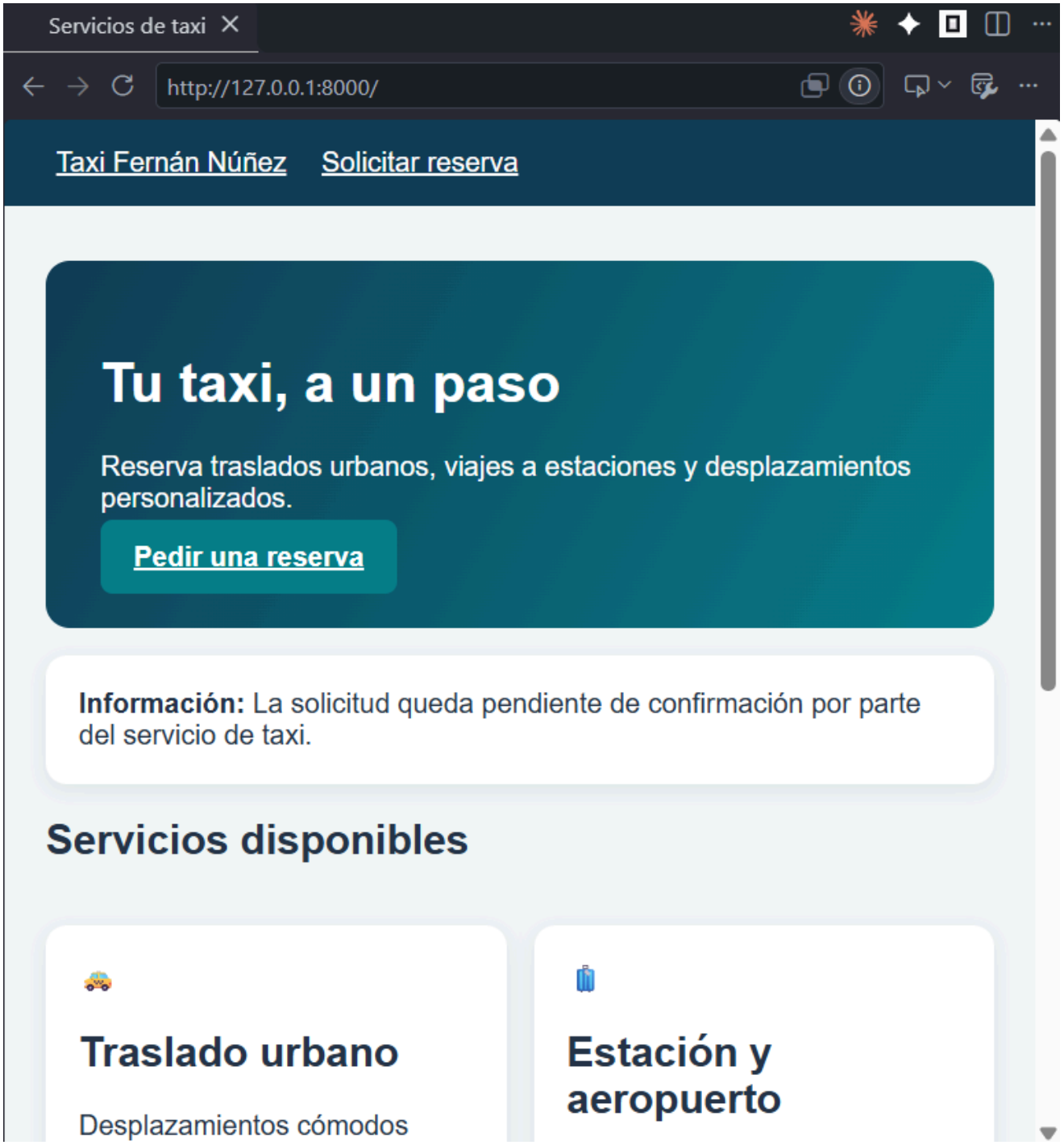
1. Usamos composer para crear el proyecto: "composer create-project laravel/laravel taxi-reservas"
2. Creamos la estructura de las carpetas y archivos que necesita

Archivo	Pista
app.blade.php	Plantilla que contiene la estructura común de las páginas.
nav.blade.php y footer.blade.php	Fragmentos reutilizables incluidos desde otras vistas.
aviso-servicio.blade.php	Otro fragmento reutilizable para mostrar un aviso común.
servicio-taxi.blade.php y alerta-taxi.blade.php	Componentes anónimos de Blade.
inicio.blade.php y contacto.blade.php	Vistas de las páginas.
Archivo de rutas web	Busca dónde se declaran las URL atendidas por la aplicación.

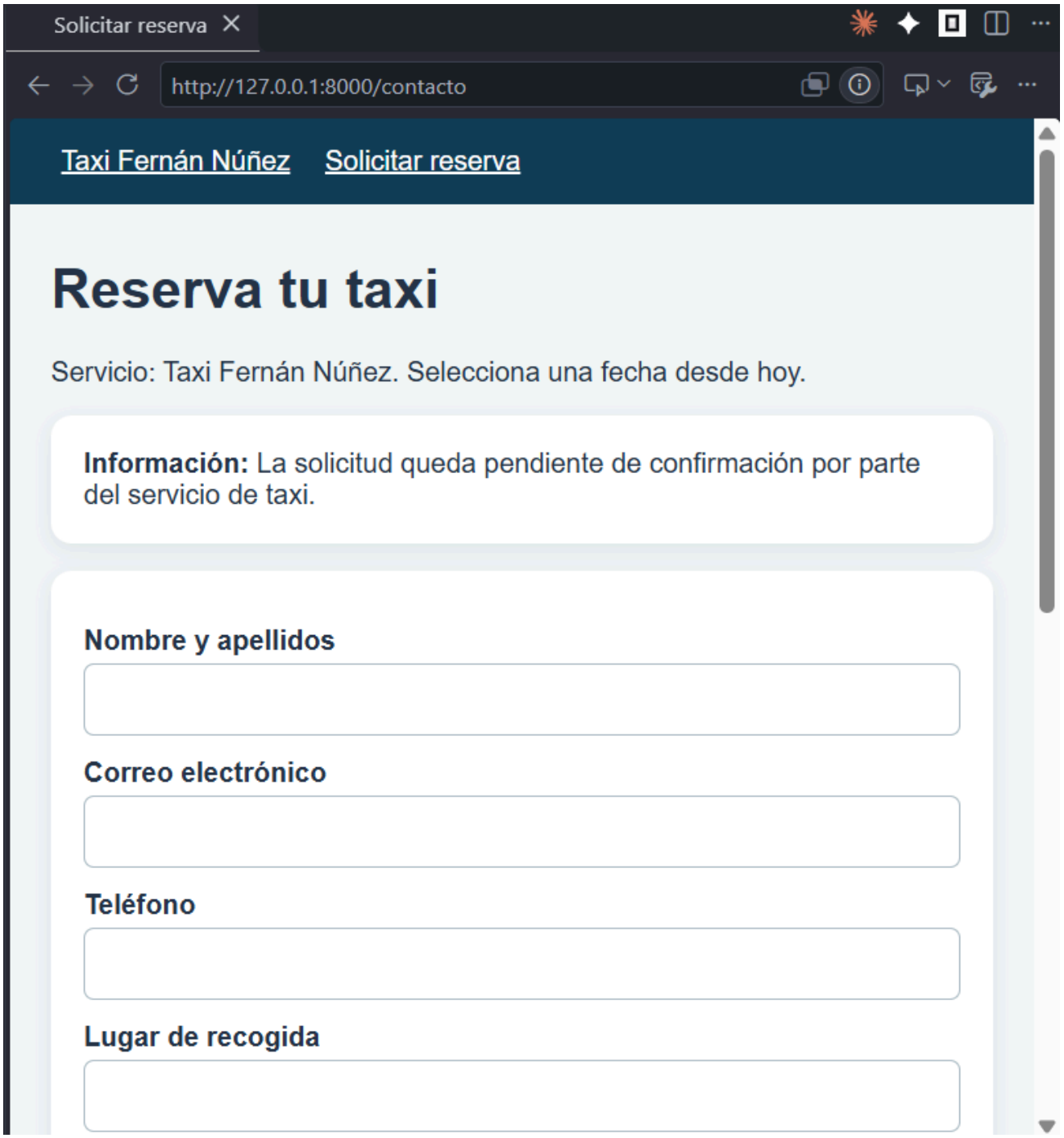


Archivos creados y completados

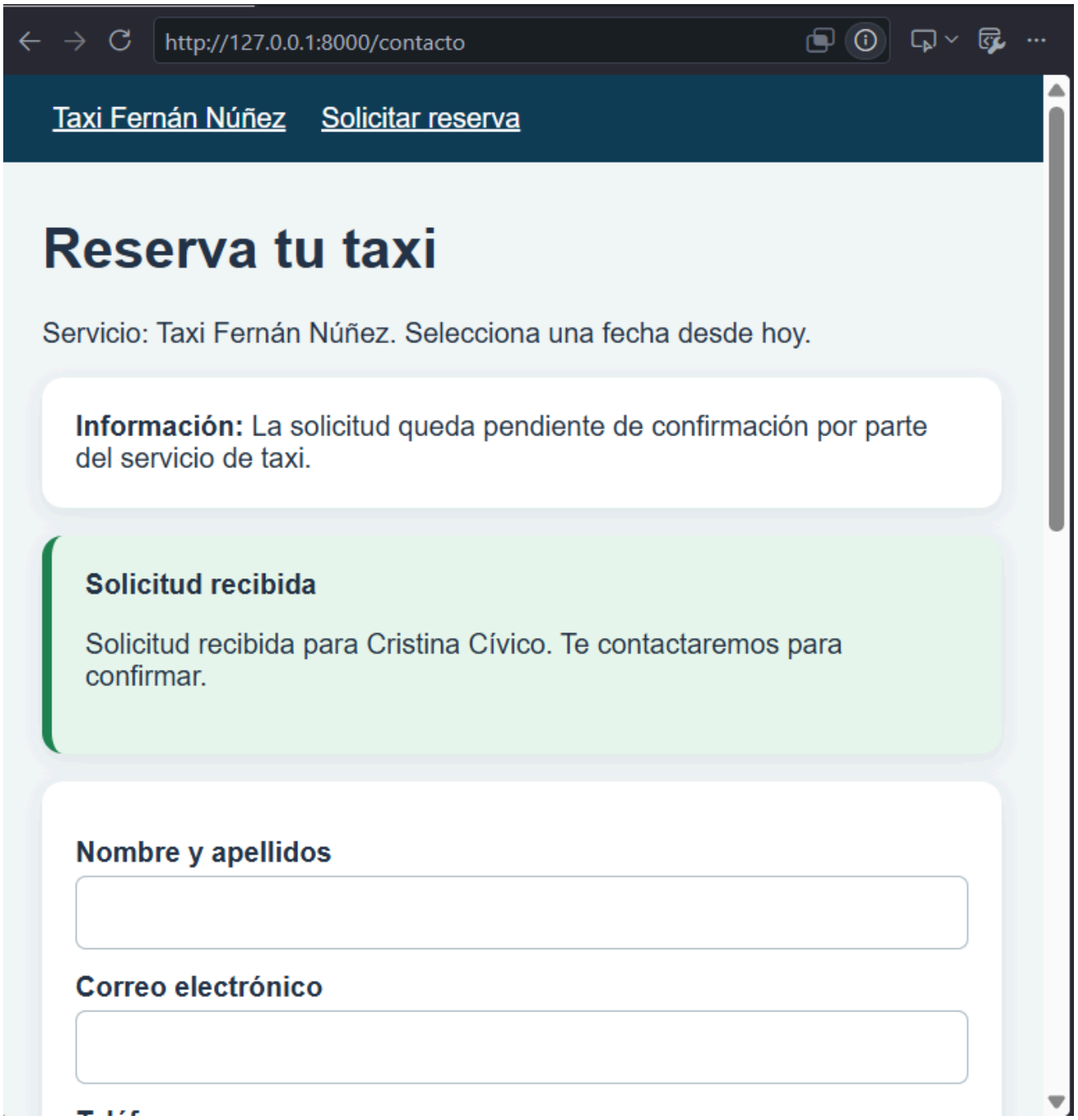
1. Página de inicio



2. Formulario de reserva



3. Confirmación de la solicitud



Cómo se conectan rutas, vistas, layout, parciales y componentes

El archivo routes/web.php es el punto de entrada. Cada ruta es una función anónima que prepara los datos y devuelve una vista con view().

```php
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    $servicios = [
```

```
        ['nombre' => 'Traslado urbano', 'precio' => '8', 'icono' => '🚕',
'descripcion' => 'Desplazamientos cómodos dentro de Fernán Núñez.'],
        ['nombre' => 'Estación y aeropuerto', 'precio' => 'Consultar', 'icono' =>
'🧳', 'descripcion' => 'Planifica tus viajes y conexiones.'],
        ['nombre' => 'Servicio a medida', 'precio' => 'Consultar', 'icono' => '📍',
'descripcion' => 'Indica el origen y el destino que necesitas.'],
    ];

    return view('inicio', compact('servicios'));
})->name('inicio');

Route::get('/contacto', function () {
    $fechaMinima = now()->format('Y-m-d');

    return view('contacto', ['fechaMinima' => $fechaMinima])
        ->with('empresa', 'Taxi Fernán Núñez');
})->name('contacto');

Route::post('/contacto', function (Request $request) {
    $datos = $request->validate([
        'nombre' => ['required', 'string', 'min:2', 'max:80'],
        'email' => ['required', 'email', 'max:150'],
        'telefono' => ['required', 'string', 'min:9', 'max:20'],
        'origen' => ['required', 'string', 'max:120'],
        'destino' => ['required', 'string', 'max:120'],
        'fecha' => ['required', 'date', 'after_or_equal:today'],
        'hora' => ['required', 'date_format:H:i'],
        'pasajeros' => ['required', 'integer', 'between:1,8'],
        'detalles' => ['nullable', 'string', 'max:500'],
    ]);

    return redirect()->route('contacto')
        ->with('reservaOk', 'Solicitud recibida para ' . $datos['nombre'] . '. Te
contactaremos para confirmar.');
})->name('reservas.store');
```

- Vistas y layout. Las vistas inicio y contacto empiezan con @extends('layouts.app'), de modo que heredan la estructura HTML común.
- Parciales. Son fragmentos reutilizables que se insertan con @include. El layout incluye partials.nav y partials.footer, por lo que el menú y el pie aparecen en todas las páginas sin repetir código.
- Componentes anónimos. Son archivos de resources/views/components/ que se usan como etiquetas <x-...> y no necesitan clase PHP.

Comprobaciones realizadas

- ☑ La página de inicio muestra las tarjetas de servicio.
- ☑ Los enlaces del menú llegan a las páginas correctas.

- ☑ El formulario contiene el token CSRF y envía los datos por POST.
- ☑ Si faltan datos, se muestran errores y se conservan los valores.
- ☑ Si los datos son válidos, aparece una confirmación. No hay controladores ni conexión a base de datos.